



nextwork.org

Deploy Backend with Kubernetes



Kehinde Abiuwa

```
[ec2-user@ip-172-31-20-159 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
[ec2-user@ip-172-31-20-159 manifests]$
```

i-Odbb84502867a6a07 (nextwork-eks-instance) ×

PublicIPs: 51.20.128.54 PrivateIPs: 172.31.20.159

[CloudShell](#) [Feedback](#) © 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)



Introducing Today's Project!

In this project, I will set up the backend for deployment, install kubectl, deploy the backend to a Kubernetes cluster, and track the rollout in EKS, because I want a running, observable backend service on Kubernetes that I can monitor and showcase end-to-end DevOps skills.

Tools and concepts

I used Kubernetes (Amazon EKS), Amazon ECR, kubectl, eksctl, and EC2 to create the cluster, push my backend image, map IAM access, and apply Deployment/Service YAML to run and expose the app. Key concepts include using manifests to declare desired state (Deployments & Services), label selectors to route traffic to Pods, replicas for high availability, rolling updates/rollbacks, and Service types (ClusterIP/NodePort/LoadBalancer)

Project reflection

This project took me approximately 40 minutes. The most challenging part was setting up IAM/EKS access so kubectl from EC2 could talk to the cluster and making sure the Service selector/ports matched the Deployment. My favourite part was watching EKS schedule 3 replicas, pull the image from ECR, and start cleanly



Backend Deployment!

To deploy my backend application, I installed kubectl, pointed it at my EKS cluster, and applied my manifests with `kubectl apply -f flask-deployment.yaml` and `kubectl apply -f flask-service.yaml`, because that created the Pods from my Deployment and a Service to route traffic to them so the app is reachable

kubectl

kubectl is the Kubernetes CLI that talks directly to the cluster's API server to create, read, update, and delete resources (e.g., Deployment, Service, ConfigMap, Pod). I need this tool to apply my manifests, inspect and debug, and verify the rollout. I can't use eksctl for the job because eksctl is mainly for provisioning and managing EKS clusters. It doesn't replace day-to-day workload operations like applying YAML or interacting with live resources.



```
[ec2-user@ip-172-31-20-159 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
[ec2-user@ip-172-31-20-159 manifests]$
```

i-0dbb84502867a6a07 (nextwork-eks-instance)

PublicIPs: 51.20.128.54 PrivateIPs: 172.31.20.159

 CloudShell [Feedback](#)

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)



Verifying Deployment

My extension for this project is to use the EKS console to monitor my cluster (nodes, Pods, events) and create an access entry for my EC2 instance. I had to set up IAM access policies because EKS ties AWS IAM to Kubernetes RBAC—without a mapping, kubectl isn't allowed. I set up access by running this in the EC2 Instance Connect tab:

```
eksctl create iamidentitymapping --cluster nextwork-eks-cluster --arn  
arn:aws:iam::511239431868:user/kehindeabiuwa-IAM-Admin --group system:masters  
--username admin --region eu-north-1
```

Once I gained access into my cluster's nodes, I discovered pods running inside each node. Pods are the smallest deployable units in Kubernetes—wrappers around one or more tightly-coupled containers that are scheduled together on the same node and managed as a single unit (via a Deployment/ReplicaSet). Containers in a pod share the same network namespace (one IP and port space, talk over localhost), the same storage volumes, and the same lifecycle/scheduling (they start/stop together).

The EKS console shows you the events for each pod, where I could see it was Scheduled, the node Pulling the ECR image, Pulled successfully, Created the nextwork-flask-backend container, and Started it. This validated that the Deployment worked end-to-end



Amazon Elastic Kubernetes Service

Dashboard [New](#)

Clusters

▼ Settings

Dashboard settings [New](#)

Console settings

▼ Amazon EKS Anywhere

Enterprise Subscriptions

▼ Related services

Amazon ECR

AWS Batch

Documentation [🔗](#)

Labels (2)

Key	Value
app	nextwork-flask-backend
pod-template-hash	76d8bcc797

Annotations (0)

No annotations

No annotations have been defined for this resource.

Events (5)

Type	Reason	Event time	From	Message
Normal	Scheduled	15 minutes ago	default-scheduler	Successfully assigned default/nextwork-flask-backend-76d8bcc797 to ip-10-0-1-10
Normal	Pulling	15 minutes ago	kubelet	Pulling image "511239431868.dkr.ecr.eu-north-1.amazonaws.com/nextwork-flask-backend"
Normal	Pulled	15 minutes ago	kubelet	Successfully pulled image "511239431868.dkr.ecr.eu-north-1.amazonaws.com/nextwork-flask-backend"
Normal	Created	15 minutes ago	kubelet	Created container: nextwork-flask-backend
Normal	Started	15 minutes ago	kubelet	Started container nextwork-flask-backend



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

